Customer SPA on the order module

Report of the work done on branch feat/gateway-api, 11 October 2026

Status	Done and verified; committed with this report.
Scope	The trade screens of the customer SPA (`gateway-api`: auction panel, buy box, orders list, order detail) now run against the order module as built, through the customer edge's existing routes, instead of a sketch of the contract and local mock data. Two pieces a member could not do were added: setting up the payout account and booking the carrier. No edge or backend change.
Tests	70 SPA tests in 15 files pass (50 in 10 before); type check and translation parity green; the customer edge jar built and driven headless with two members against the order module and announce-management, with the provider and carrier webhooks signed by the driver; checklist in section 5.

1. What it is about

A member buys a fixed-price announce (the seller accepts, the buyer pays through the payment provider, the seller books the carrier, the carrier reports, the buyer confirms receipt, the payout is released and the deal settles) or bids on an auction. The order module holds this as four contexts (auction, order, payment, shipping) with their own read-model rows and rules; it refuses to accept a purchase, and never opens an auction, until the seller has a payout account with the provider.

The customer screens were written on 26 September against the sketch in doc/order-context.md section 8, before the module existed, and only ran on the service worker's fixtures. The customer edge has routed the member routes to the module since then, so the work was a comparison of the client with the module as built, the corrections it called for, the two missing pieces, and a run against the service.

2. What the comparison found and what the SPA does now

Found	Now
Cents and sketch names; ten order statuses; auctions closing as CLOSED	The read-model rows field for field (decimals, title, agreedPrice, feeAmount, payoutAmount, currentPrice, bidIncrement, endTime); thirteen order statuses and the four auction outcomes, each named and coloured.
The auction read taken as the row; the orders list as a page with a timeline	The auction is {auction, bids} and the minimum next bid comes from the row; the orders list is a plain array; the detail combines the order, payment and shipment rows (404 = none yet) and derives its timeline from their timestamps, as the admin SPA does.
Commands treated as answering the row	They answer the aggregate in cents; the screens use its fields where helpful (the payouts card shows the answer at once) and refetch the rows.
No way to set up payouts, so a seller could never accept	A payouts card on the seller tab of the orders page and on the profile; a seller-not-payable refusal links to it.
A "Pay (fake provider)" button that posted the webhook from the browser	Removed: the webhook is HMAC-signed with the backend's secret and the real flow is the provider's checkout with the payment row's client secret. The detail shows the payment state and "waiting for your payment through ".
An empty shipment request	The seller books with service, parcel size and the two addresses; the shipment card shows the tracking number, the carrier's last event and, for the seller, the label link.
Rows read right after a command, or right after Buy navigated to the new order	A refetch a second later after each command, and a quiet retry of a missing row (seen: the seller account row landed 1 s after the command).

Flags: `trade` defaults on; both phase-2 flags do now. Mocks: `app/mocks/tradeData.ts` in the rows' shapes with the module's rules (409 own-announce, not-proposed, already-paid, seller-not-payable, bid-below-minimum, seller-cannot-bid, auction-not-open, not-paid; 403 not-seller, not-buyer, not-party; 404 for a payment or shipment that does not exist yet; the fee computed at accept), serving the browser with `features.mocks` and the page tests.

3. What changed

Area	Files (under gateway-api/src/main/webapp)
Client and types	`app/shared/model/trade.ts` (rows, requests, aggregate answers, helpers), `app/shared/api/tradeApi.ts`, `app/shared/util/orderTimeline.ts`, `app/shared/util/settle.ts` (settle delay and quiet retries, shared with the chat thread)
Screens	`app/modules/trade/AuctionPanel.tsx`, `BuyBox.tsx`, `Orders.tsx`, `OrderDetail.tsx`, `PayoutsCard.tsx` (new), `app/modules/auth/Profile.tsx` (payouts card)
Flags and mocks	`app/config/features.ts` (trade on), `app/mocks/tradeData.ts` (new, replaces data.ts), `handlers.ts`
Tests	specs for AuctionPanel (6), BuyBox (2), Orders (3), OrderDetail (7), orderTimeline (2); `app/test/testUtils.tsx` seeds the trade mocks and can render a visitor
Strings	`app/i18n/locales/{en,da,sv}/trade.json` rewritten: statuses, doors, payment and shipment states, timeline steps, payouts
Docs	`doc/frontend-edges.md` section 16 and 16a; this report

4. Verification

- `npm test`: 70 tests in 15 files pass. `npm run typecheck` and `npm run i18n:check` clean.
- The customer edge jar was built with the webapp profile and driven headless (Playwright) with two freshly registered members; the provider and carrier webhooks were posted by the driver, signed with the fake secrets, through the edge's public webhook routes.
- The order module and announce-management ran from jars built with the framework module in the same reactor (see section 6).

5. Checklist, executed 11 October 2026

Setup: live MySQL, Kafka 9094, customer edge on 8092, announce-management on 8085 and the order service on 8083 from their jars, service tokens in enforce. Deviations: the development EventSourcingDB on port 3000 still hangs on streaming reads, so both backends ran against a throwaway store on port 3001 with their nine read-side checkpoint rows deleted before and restored after; announce-management with the search indexer off. Everything the runs wrote was deleted from the development database afterwards.

Step	Result
Seller sets up payouts in the SPA	"Payouts enabled · FAKE · VERIFIED" at once from the aggregate's answer; the row answers a moment later; the gate rows of the order and auction contexts followed in 18 ms (1 s and about 90 s in two earlier runs)
Seller publishes a fixed-price and an auction announce	the auction row is OPEN 1.3 s after publishing (start 100, increment 10)
Buyer clicks Buy	201, lands on the PROPOSED order at the first attempt; "Waiting for the seller to accept", Cancel, no payment or shipment yet
Seller accepts	"Awaiting payment", payment card "awaiting payment · 1.250 kr. · FAKE · Reference pi_fake_1"; the buyer's screen follows by polling in 5.8 s with "Waiting for your payment through FAKE."

Step	Result
Provider webhook CONFIRMED	204; buyer sees Paid in 9.9 s, payment "confirmed", dispute form shown
Seller books the carrier	shipment card "FAKE · MyPack Collect FAKE00000001 · Booked" with the label link; the row REQUESTED with service and parcel size
Carrier webhooks DISPATCHED, DELIVERED	204 ×2; Shipped in 8.9 s, Delivered in 9.9 s on the buyer's screen; no label link for the buyer
Buyer confirms receipt	"Releasing payout", payment row RELEASING with a payout reference; after PAYOUT_SENT, Settled in 8.4 s; timeline Agreed, Paid, Shipped, Delivered, Receipt confirmed, Payout released, Settled; payment "paid out"; no button left
Auction: bid below the minimum (REST)	409 bid-below-minimum with the module's detail
Auction panel, buyer	100 kr., 0 bids, countdown, minimum 100 (increment 10); bid 100 → "Your bid: 100 kr. (leading)"; bid 110 → 110 kr., 2 bids, minimum 120, "Retract my bid"; the row holds 110 ACTIVE and 100 OUTBID
Auction panel, owner	"This is your auction.", no bid input
Seller buys their own announce (REST)	409 (announce-not-available, ahead of own-announce)
Console	the expected 401 of the silent refresh before login and the 404s of the payment and shipment reads before they exist and of the quiet retries

6. Gotchas met on the way

- Shared ~/.m2 again: at 15:52 another worktree installed a `cqrs-reactive` without `security.Callers`; the order service and announce-management started from it hung on the first member request (NoClassDefFoundError inside the handler, no answer through the edge). Packaging both with the framework in the same reactor fixed it for good.
- Ports 8090 and 8091 were held by the achatglasses stack of another session; the first drive registered two members through its edge into that stack's database (removed there) before moving to 8092.
- The seller account row, the gate rows and a just-created order land a moment after their commands answer; refetching at once showed stale rows until the settle-and-retry pattern of the chat round was applied here too.
- When the order module came up it consumed the chat round's OfferAcceptedV1 from messaging.v1 and created an order through the offer door; removed with the run's rows.
- `pgrep -f` with a pattern that appears later in the same command line kills the calling shell; kill by saved PID.

7. Left open

- A real payment provider and its checkout on the payment row's client secret.
- Live updates for trade (the screens poll every 5 s / 15 s); the order module exposes no push channel.
- `GET /api/auctions/{id}` for visitors: a one-line PublicPaths change on the edge would show the auction state before login.
- Auction closing from the SPA, proxy bidding, refund and dispute resolution screens on the customer side.
- The development EventSourcingDB on port 3000 needs a container restart before any backend can use it again.